

AUDITORIA TECNICA

Plataforma Bateu

Relatorio de Auditoria Completa

Analise profunda de arquitetura, seguranca, qualidade de codigo, desempenho e conformidade regulamentar da plataforma de entretenimento e jogos.

Escopo: Analise de 200+ ficheiros de codigo-fonte, 90+ componentes de jogos ao vivo, sistema de autenticao RBAC com 5 perfis, 60+ migracoes SQL, e integrações com Supabase, PayPal e Stripe.

Data: Agosto 2026

Versao: 2.0

Classificacao: Confidencial

Índice

1. Resumo Executivo
2. Arquitectura e Organização do Código
3. Segurança e Autenticação
4. Qualidade do Código e Padrões
5. Sistema de Jogos ao Vivo
6. Internacionalização e Acessibilidade
7. Desempenho e Otimizações
8. Infraestrutura de DevOps e Base de Dados
9. Sistema de Gestores Regionais
10. Recomendações Prioritárias
11. Conclusão

1. Resumo Executivo

RESULTADO GERAL: REQUER ATENCAO

A plataforma Bateu (bateu.online) é um ecossistema abrangente de entretenimento digital, rifas, concursos, jogos ao vivo e funcionalidades de negocios. Construída com React (Vite), TypeScript, Tailwind CSS e Supabase como backend-as-a-service, a plataforma apresenta uma arquitetura ambiciosa com mais de 200 ficheiros de código-fonte, 90+ componentes de jogos ao vivo, 60+ migrações de base de dados e um sistema RBAC com cinco perfis de utilizador distintos (user, business, admin, superadmin, regional_manager).

Esta auditoria avaliou a plataforma em seis dimensões críticas: arquitetura e organização do código, segurança e autenticação, qualidade do código e padrões, desempenho e otimizações, internacionalização e acessibilidade, e infraestrutura de DevOps. A análise revelou que, embora a plataforma demonstre uma visão de produto notável e uma quantidade impressionante de funcionalidades, existem áreas significativas que requerem atenção imediata, particularmente no que diz respeito à segurança de jogos com lógica cliente-side, fragmentação de estado de autenticação, e cobertura incompleta de internacionalização.

O sistema de jogos ao vivo, com 90+ componentes incluindo quiz multiplayer, batalhas de trivia, jogos de dados, e jogos de carta, é o ponto mais forte da plataforma em termos de funcionalidade. A integração com Supabase fornece autenticação, base de dados em tempo real e armazenamento, enquanto os gateways de pagamento PayPal e Stripe permitem monetização. O sistema de gestores regionais, recentemente implementado, adiciona capacidade de operação multi-país com branding personalizado.

Metrica	Valor	Avaliacao
Ficheiros de código (src/)	200+	Alta complexidade
Componentes de jogos ao vivo	90+	Impressionante
Migrações SQL	60+	Bem estruturado
Rotas definidas (App.tsx)	80+	Muito extenso
Componentes de UI (shadcn)	40+	Boa base
Idiomas suportados (i18n)	6 (en, pt, pt-BR, es, fr, hi)	Parcial (13% adoção)
Perfis RBAC	5 perfis	Completo

Tabela 1: Metricas gerais da plataforma

2. Arquitectura e Organizacao doCodigo

2.1 Stack Tecnologico

A plataforma utiliza um stack moderno e bem escolhido para o seu dominio. O frontend e construido com React 18 e TypeScript, empacotado com Vite para tempos de build rapidos. O sistema de estilizacao combina Tailwind CSS para utilitarios rapidos com componentes shadcn/ui para elementos de interface consistentes. A gestao de estado e distribuida: React Context para autenticao, idioma, tema e configuracao regional, e TanStack Query (React Query) para cache e sincronizacao de dados com o backend.

O backend e inteiramente fornecido pelo Supabase, que oferece autenticao (email/password e OAuth), base de dados PostgreSQL com Row Level Security (RLS), armazenamento de ficheiros, e subscriptions em tempo real via WebSockets. Esta abordagem backend-as-a-service permite desenvolvimento rapido, mas introduz dependencia critica num fornecedor unico. A ausencia de uma camada de API intermedia (serverless functions ou microservicos) significa que toda a logica de negocio sensivel reside no cliente, o que e um risco significativo de seguranca detalhado na seccao de seguranca.

2.2 Estrutura de Directorios

A organizacao de directorios segue uma estrutura baseada em funcionalidades (feature-based), geralmente considerada uma boa pratica. Os componentes estao organizados em pastas tematicas: components/livegames/ para jogos ao vivo (90+ ficheiros), components/ui/ para componentes de interface base (40+ ficheiros shadcn), components/wallet/ para funcionalidades de carteira, components/payments/ para gateways de pagamento, e components/mobile/ para navegacao movel. As paginas estao divididas em pages/ para paginas de nivel superior, pages/admin/ para o painel de administracao, pages/dashboard/ para o painel de negocios, pages/esports/ para a seccao de esports, e pages/tournaments/ para torneios.

A pasta contexts/ contem quatro provedores de contexto (Auth, Language, Theme, Currency) e o hook useRegionalConfig.tsx para configuracao regional. A pasta integrations/supabase/ contem o cliente tipado e as definicoes de tipos gerados. As migracoes SQL em supabase/migrations/ totalizam mais de 60 ficheiros, demonstrando uma evolucao activa do schema de base de dados. Um ponto positivo e a presenca de layouts separados (DashboardLayout.tsx e AdminLayout.tsx) para diferentes areas da aplicacao, melhorando a separacao de preocupacoes.

2.3 Roteamento e Navegacao

O ficheiro App.tsx centraliza todo o roteamento da aplicacao com mais de 80 rotas definidas. A aplicacao usa React Router v6 com rotas aninhadas para o painel de negocios (/dashboard/*) e o painel de administracao (/admin/*). A protecao de rotas e implementada pelo componente ProtectedRoute, que suporta controle baseado em roles (requiredRole) e bloqueio de roles especificos (blockRoles). O sistema suporta cinco perfis de utilizador, e a logica de protecao verifica cada role individualmente, com tratamento especial para superadmin ter acesso a rotas

de admin.

A navegacao inclui transicoes de pagina com Framer Motion (AnimatePresence + PageTransition), uma barra de navegacao inferior para dispositivos moveis (BottomTabBar), um gaveta de menu lateral (MobileMenuDrawer), e um rastreador de paginas recentes (RecentPagesTracker). A experiencia de overlay para transmissoes ao vivo exclui intencionalmente os elementos de navegacao para maximizar a area de visualizacao. Um ponto de atencao e que o App.tsx e um ficheiro extenso com 462 linhas e 148 importacoes, sugerindo que a separacao em modulos menores melhoraria a manutenibilidade.

3. Seguranca e Autenticacao

CRITICO - RISCOS ELEVADOS IDENTIFICADOS

3.1 Sistema de Autenticacao

O sistema de autenticacao e gerido pelo AuthContext, que envolve o Supabase Auth para signup, signin e signout. O fluxo de autenticacao e bem estruturado com uma maquina de estados que gerencia user, session, profile, role, adminCountries e loading. O contexto detecta automaticamente sessoes OAuth e cria perfis basicos para utilizadores que saltam o registo tradicional. A resolucao de roles segue uma prioridade clara: superadmin > admin > regional_manager > business > user, garantindo que o role mais privilegiado prevalece quando um utilizador tem multiplas roles atribuidas.

O processo de registo suporta metadados adicionais (display_name, role, company_name) passados durante o signup. Dados extras como telefone, provincia, cidade e interesses sao armazenados no localStorage e processados apos o primeiro login, garantindo que nao sao perdidos se o utilizador fechar a janela durante o registo. O sistema de referral e processado automaticamente no primeiro login, com atribuicao de 50 pontos Lucky Points tanto para o referenciador como para o novo utilizador, usando um codigo de referral armazenado no localStorage.

3.2 Protecao de Rotas (RBAC)

O componente ProtectedRoute implementa um sistema de controle de acesso baseado em roles que funciona em tres modos: sem parametros (apenas exige autenticacao), com requiredRole (exige um role especifico com tratamento especial para superadmin ter acesso a rotas de admin), e com blockRoles (impede utilizadores com certos roles de aceder a uma rota). A implementacao e robusta e cobre todos os cinco perfis de utilizador. As rotas de administracao exigem role admin com superadmin tendo acesso automatico, as rotas de negocios exigem role business, e as rotas de gestor regional exigem role regional_manager com admin e superadmin tambem tendo acesso.

Uma limitacao identificada e que o ProtectedRoute nao implementa protecao baseada em regio (country-level access control). O AuthContext carrega adminCountries, mas esta informacao nao e usada pelo ProtectedRoute nem por qualquer middleware de rota. Isto significa que um gestor regional de Mocambique pode aceder a dados de Angola sem restricao no frontend, dependendo unicamente das politicas RLS do Supabase para filtragem de dados. Recomenda-se implementar verificacao de regio no ProtectedRoute para adicionar uma camada adicional de seguranca.

3.3 Riscos de Seguranca Criticos

O maior risco de seguranca da plataforma reside na ausencia de uma camada de API server-side. Com a arquitetura actual, o cliente comunica directamente com o Supabase, significando que toda a logica de negocio e visivel e modificavel no frontend. Embora o Supabase ofereca Row

Level Security (RLS) para protecao ao nivel da base de dados, existem muitas operacoes sensiveis que deveriam ser validadas server-side antes de atingir a base de dados.

Operacoes financeiras como depositos, levantamentos e apostas devem ser processadas numa camada server-side (Supabase Edge Functions ou backend separado) que valide valores minimos e maximos, verifique saldos suficientes, aplique limites de taxa, e registre auditoria. A carteira digital (Wallet) e os jogos que envolvem dinheiro real exigem esta protecao adicional para evitar manipulacao de valores por utilizadores mal-intencionados com acesso as ferramentas de desenvolvedor.

3.4 Gestao de Sessao e Tokens

A gestao de sessao e delegada ao Supabase Auth, que gera JWT tokens com validade configuravel. O AuthContext escuta mudancas de estado via onAuthStateChange e atualiza o estado de forma reactiva. O bootstrap da sessao usa um padrao defensivo com um flag bootstrapped para evitar processamento duplicado de eventos durante a inicializacao. O signout limpa todo o estado local incluindo user, session, profile e role. Uma melhoria seria adicionar verificacao de expiracao de sessao mais agressiva no cliente e invalidacao de sessoes em caso de mudanca de password.

Risco	Severidade	Area	Recomendacao
Sem API server-side	Critico	Arquitectura	Implementar Edge Functions ou backend
Jogos com logica cliente-side	Alto	Jogos	Mover logica de jogo para servidor
Operacoes financeiras no cliente	Alto	Wallet	Validar server-side com transacoes
Sem protecao de regioao nas rotas	Medio	RBAC	Adicionar verificacao de country
Referral via localStorage	Baixo	Auth	Mover para cookie ou server-side
Signup extra em localStorage	Baixo	Auth	Usar Supabase metadata

Tabela 2: Principais riscos de seguranca identificados

4. Qualidade do Código e Padrões

4.1 Tipagem e TypeScript

A plataforma utiliza TypeScript de forma consistente, com interfaces bem definidas para os modelos de dados principais. O `AuthContext` define interfaces claras para `Profile` e `AuthContextType`. Os componentes de jogos usam interfaces de estado tipadas. A pasta de tipos do Supabase contém tipos gerados automaticamente. No entanto, o uso de coerções de tipo em várias partes do código, nas inserções de perfis e referrals, indica que os tipos gerados nem sempre cobrem todos os casos de uso, sugerindo uma discrepância entre o schema da base de dados e os tipos disponíveis no cliente.

A prática de contornar o sistema de tipos elimina uma das principais vantagens do TypeScript. Quando um campo é inserido com tipo forçado, qualquer erro de mapeamento passará despercebido pelo compilador. Recomenda-se criar interfaces de input dedicadas que espelhem os campos requeridos por cada tabela, em vez de usar o tipo completo que pode incluir campos gerados automaticamente como `created_at` e `id`.

4.2 Componentes e React Patterns

Os 90+ componentes de jogos ao vivo seguem um padrão estruturado. Cada jogo é um componente funcional React com estado local gerenciado por hooks. Os jogos mais complexos como o quiz multiplayer, batalhas de trivia, e o jogo do milionário demonstram domínio de React patterns avançados incluindo gerenciamento de efeitos colaterais, memoização de callbacks, e refs para intervalos e timers de jogo.

O componente `SafeGameLoader` implementa error boundaries para jogos, impedindo que um jogo com erro crash toda a aplicação. Este padrão é essencial para uma plataforma com mais de 90 componentes de jogos, onde a probabilidade de erros em componentes individuais é alta. O componente exibe uma interface de erro amigável com opção de selecionar outro jogo, em vez de uma tela branca ou crash silencioso. Este nível de resiliência demonstra maturidade na arquitetura de componentes.

O sistema de provedores de contexto segue o padrão `Provider` do React. A hierarquia de providers no `App.tsx` é profundamente aninhada com sete níveis de providers encapsulados. Embora o React 18 com modo concorrente lide melhor com isto do que versões anteriores, a profundidade da árvore de providers pode causar re-renderizações em cascata. A divisão em provedores especializados é boa para separação de preocupações, mas um provedor de configuração unificada poderia reduzir a profundidade.

4.3 Gestão de Estado

A plataforma usa uma abordagem híbrida de gestão de estado. O estado global de autenticação, idioma, tema e moeda é gerido por React Contexts. O estado de dados do servidor é gerenciado pelo TanStack Query (React Query), que fornece cache automático, revalidação em fundo, e

estados de loading/error. O estado local dos jogos é gerenciado por hooks `useState` e `useRef` dentro de cada componente. Esta separação é adequada para o domínio, mas merece atenção em dois pontos.

Primeiro, o estado de auth (user, profile, role) é armazenado em contexto React, mas não é persistido fora da sessão. Isto significa que a cada recarregamento de página, o perfil e role são re-buscados ao Supabase. Embora o cache do React Query mitigue isto parcialmente, a experiência de recarregamento poderia ser melhorada com hidratação rápida a partir de dados de sessão. Segundo, o estado dos jogos (saldo, resultados, multiplicadores) é inteiramente local e perdido quando o utilizador navega para fora do jogo.

5. Sistema de Jogos ao Vivo

PONTO FORTE DA PLATAFORMA

5.1 Catalogo de Jogos

A plataforma possui um catalogo impressionante de mais de 90 componentes de jogos ao vivo organizados em multiples categorias. Os jogos incluem quiz multiplayer (KahootMultiplayerQuiz, TriviaFlash, QuizBattle), jogos de carta (TeenPatti, UnoCardGame, CheckersGame, ChessGame), jogos de dado (DiceDuel, LuckyDice, BakuganDice), jogos de palavras (WordScramble, WordChain, KeywordHunt), jogos de memoria (MemoryChallenge, MemoryCardsVS, PatternMemory), jogos de destreza (FlappyBirdGame, FruitNinjaGame, SpeedReaction, TapBattle), e jogos de estrategia (BattleshipGame, ConnectFourGame, LudoGame, Dominoes).

Alem dos jogos genericos, a plataforma inclui jogos culturalmente relevantes para o mercado africano lusofono: BichoGame (jogo do bicho), DjikotaGame, ChigogoGame, NtchuvaGame, MexericaGame, e UrusseGame. Esta adaptacao cultural e um diferencial competitivo significativo, demonstrando compreensao do publico-alvo. O sistema de premiacoes e flexivel, com suporte a rodas de premios personalizaveis (PrizeWheel) e o jogo do milionario (EnhancedMillionaireGame) com niveis de dificuldade configuraveis.

5.2 Infraestrutura de Jogos

Os jogos sao carregados dinamicamente pelo componente SafeGameLoader, que fornece uma camada de isolamento e recuperacao de erros. O LiveHub serve como pagina principal de jogos, com sistema de busca, filtragem por categoria, e suporte a branding personalizado por regio. Cada jogo recebe props de configuracao (onScore, liveCode, prizes, branding) atraves do SafeGameLoader, permitindo personalizacao sem modificar o codigo do jogo.

O sistema suporta jogos single-player e multiplayer. Os jogos multiplayer usam subscriptions em tempo real do Supabase para sincronizar estado entre jogadores. O LiveControlPanel fornece ao anfitriao da sessao ferramentas para iniciar, parar e configurar jogos. O LiveLeaderboard exhibe classificacoes em tempo real. O sistema de batalhas (Battles) permite que dois jogadores compitam directamente em desafios de varios jogos.

Categoria	Quantida de	Exemplos
Quiz e Conhecimento	6+	Kahoot, TriviaFlash, QuizBattle, CapulanaQuiz, BichoGame
Cartas e Tabuleiro	10+	TeenPatti, Uno, Xadrez, Damas, Dominos, Ludo
Palavras e Letras	4+	WordScramble, WordChain, KeywordHunt
Destreza e Reacao	8+	FlappyBird, FruitNinja, SpeedReaction, TapBattle, WhackAMole

Categoria	Quantidade	Exemplos
Estrategia e Puzzle	8+	BatalhaNaval, Connect4, 2048, MazeRace, MemoryCards
Dados e Roleta	5+	DiceDuel, LuckyDice, ChallengeRoulette, SlotsVS
Culturais Africanos	6+	Djikota, Chigogo, Ntchuva, Mexerica, Urusse, Bicho
Entretenimento Social	8+	TruthOrDare, NeverHaveIEver, HotPotato, HotSeat, PunishmentWheel

Tabela 3: Catalogo de jogos ao vivo por categoria

6. Internacionalizacao e Acessibilidade

6.1 Sistema de Idiomas

A plataforma implementa um sistema de internacionalizacao customizado atraves do `LanguageContext`, que suporta seis idiomas: ingles (en), portugues (pt), portugues brasileiro (pt-BR), espanhol (es), frances (fr) e hindi (hi). O contexto fornece um hook `useLanguage()` com uma funcao `t(key)` que resolve chaves de traducao com fallback automatico: se a chave nao existir no idioma actual, recua para ingles. Isto permite adicionar traducoes de forma incremental sem quebrar a interface.

O `LanguageContext.tsx` e um ficheiro extenso com mais de 5300 linhas contendo todas as traducoes embutidas. O ficheiro contem chaves organizadas por seccao (nav, regional, livehub, app, etc.) com traducoes para os seis idiomas. O sistema de resolucao de idioma inclui deteccao automatica baseada no pais do utilizador (via `CountryLanguageSync`), que mapeia codigos de pais para idiomas preferidos. O utilizador tambem pode alterar manualmente o idioma atraves do componente `LanguageSwitcher`.

6.2 Cobertura de Traducao

COBERTURA PARCIAL - 13% DE ADOCAO

A principal limitacao do sistema de i18n e a baixa taxa de adocao nos componentes da aplicacao. A analise revelou que apenas 48 dos 366 ficheiros (13%) utilizam o hook `useLanguage()`. Isto significa que a grande maioria dos componentes ainda exhibe texto em portugues endurecido, tornando a plataforma inacessivel para utilizadores nao lusofonos. Os componentes que estao traduzidos incluem `LiveHub`, `RegionalManagerPanel`, `Profile`, `Login`, `HeroSection` e os nomes e descricoes dos 65 jogos.

Uma lacuna significativa e que as paginas de administracao (18 paginas em `pages/admin/`) nao estao traduzidas, limitando a usabilidade do painel de administracao a utilizadores que falam portugues. As paginas de esports (8 ficheiros), os componentes de pagamento (4 ficheiros), e os componentes de wallet (5 ficheiros) tambem nao estao traduzidos. Recomenda-se priorizar a traducao dos componentes de wallet e pagamento, seguidos pelas paginas de admin, pois estes sao os mais criticos para a operacao da plataforma.

Area	Ficheiros	Traduzidos	Cobertura
Componentes de jogos (livegames/)	90+	65 nomes/desc	Nomes apenas
Paginas de admin	18	1	6%
Dashboard de negocios	24	3	13%
Componentes de UI base	40+	2	5%
Wallet e pagamentos	9	0	0%

Area	Ficheiros	Traduzidos	Cobertura
Esports	8	0	0%
Componentes publicos	20+	5	25%

Tabela 4: Cobertura de traducao por area da aplicacao

7. Desempenho e Otimizacoes

7.1 Carregamento Inicial

A plataforma implementa uma tela de carregamento (LoadingScreen) que e exibida durante os primeiros 4 segundos da aplicacao, ou 1.8 segundos apos o carregamento da configuracao regional. Esta abordagem de loading fixo baseada em tempo nao e ideal, pois pode mostrar a tela de carregamento desnecessariamente em dispositivos rapidos ou esconde-la prematuramente em dispositivos lentos. Recomenda-se usar um sistema de loading baseado em eventos, onde a tela e removida apenas apos a conclusao de todas as chamadas de inicializacao criticas.

O App.tsx importa 148 modulos no nivel superior, o que significa que todos eles sao incluidos no bundle inicial. Embora o Vite faca code splitting automatico para rotas com importacao dinamica, as importacoes estaticas de 90+ componentes de jogos e 40+ componentes de UI significam que o bundle inicial e significativamente grande. A maioria dos componentes de jogos so e utilizada na pagina LiveHub, mas sao importados no App.tsx, impedindo o tree shaking. Recomenda-se converter todas as importacoes de jogos para importacao dinamica (React.lazy + Suspense).

7.2 Componentes de Interface

A plataforma inclui uma camada de componentes de UI baseada no shadcn/ui, que fornece componentes acessiveis e bem otimizados. Os componentes de effects (GlowEffects, ParticleBackground, ConfettiSystem, LottieAnimation) adicionam polimento visual, mas podem impactar o desempenho se usados excessivamente. O ParticleBackground em particular, se activo em todas as paginas, consumiria recursos de GPU significativos em dispositivos moveis. Recomenda-se desactivar animacoes pesadas em dispositivos de baixo desempenho usando media queries ou a API de deteccao de hardware.

O componente BackgroundDecorations e renderizado em todas as paginas excepto overlays. Este componente provavelmente inclui elementos decorativos com posicao absoluta que, embora nao interactivos, ocupam memoria de composicao do navegador. A simplificacao destes elementos decorativos ou a sua conversao para CSS puro (gradientes, sombras, borders) reduziria o custo de renderizacao.

7.3 Consultas de Dados

A plataforma usa TanStack Query (React Query) para gerenciamento de estado do servidor, o que e uma escolha excelente. O React Query fornece cache automatico, deduplicacao de pedidos, revalidacao em fundo, e invalidacao de cache. No entanto, a eficacia depende de como as queries sao configuradas. As queries devem ter staleTime e gcTime configurados adequadamente para evitar pedidos desnecessarios ao Supabase, que tem limites de requisicoes no plano gratuito.

O componente AuthContext faz multiplas consultas sequenciais ao Supabase durante a inicializacao: busca o perfil, depois busca os roles, depois busca as regioes admin, depois

processa dados de referral. Estas quatro consultas poderiam ser paralelizadas usando Promise.all, reduzindo o tempo de inicializacao. Alem disso, os resultados de roles e regioes admin raramente mudam e poderiam ter cache mais agressivo.

8. Infraestrutura de DevOps e Base de Dados

8.1 Migracoes SQL

A plataforma possui mais de 60 ficheiros de migracao SQL em `supabase/migrations/`, demonstrando uma evolucao activa e bem documentada do schema da base de dados. As migracoes estao organizadas cronologicamente e cobrem todas as funcionalidades da plataforma: tabelas de rifas, concursos, pagamentos, referrals, pontos de lealdade, jogos ao vivo, ligas, esports, e o sistema de gestores regionais. A maioria das migracoes inclui comentarios explicativos e operacoes idempotentes (`IF NOT EXISTS, CREATE TABLE IF NOT EXISTS`).

O sistema de gestores regionais possui uma migracao consolidada (`20260811_complete_regional_managers_system.sql`) que cria todas as tabelas, politicas RLS, indexes e funcoes auxiliares numa unica migracao idempotente. Esta migracao inclui 11 partes: alteracao do tipo enum, criacao de tabelas, adicao de colunas, insercao de dados iniciais, configuracao de RLS com 8 politicas, criacao de 10 indexes de performance, 3 triggers de `updated_at`, 4 funcoes auxiliares, e dados de seed para 12 regioes africanas. A consolidacao de multiplas migracoes parciais numa unica migracao de referencia e uma boa pratica para novos ambientes.

8.2 Row Level Security

As migracoes SQL demonstram que politicas RLS (Row Level Security) estao configuradas para as tabelas principais. As politicas seguem um padrao consistente: utilizadores autenticados podem ler dados publicos, utilizadores so podem modificar os seus proprios dados, e admins podem modificar dados da sua regioao. O sistema de gestores regionais usa funcoes auxiliares como `is_regional_manager_for_region()` e `get_my_managed_regions()` para implementar controle de acesso ao nivel de regioao directamente na base de dados.

Um ponto de atencao e que as politicas RLS sao a unica linha de defesa para operacoes de escrita, dado que nao existe API server-side. Isto significa que toda a logica de validacao de negocio deve estar codificada em triggers e constraints da base de dados, que sao mais dificeis de testar e manter do que codigo de aplicacao. A verificacao de saldos suficientes para apostas, por exemplo, deve ser feita por um trigger ou constraint, nao pelo codigo do cliente.

8.3 Processos Agendados

A plataforma possui uma pagina de administracao para cron jobs (AdminCronJobs), sugerindo a existencia de processos agendados para tarefas como sorteio de rifas, expiracao de bilhetes, e limpeza de dados. A implementacao especifica destes cron jobs nao foi analisada em detalhe nesta auditoria por estar fora do alcance do codigo-fonte frontend, mas a presenca de uma interface de gestao indica que existe alguma infraestrutura de processos em lote.

9. Sistema de Gestores Regionais

FUNCIONALIDADE RECENTE - BEM ESTRUTURADO

O sistema de gestores regionais é uma funcionalidade recentemente implementada que permite operar a plataforma em múltiplos países com configuração independente por região. O sistema inclui uma tabela `regions` com 12 regiões africanas pre-configuradas (Mocambique, Angola, Brasil, Portugal, Cabo Verde, etc.), cada uma com moeda, fuso horário e idioma próprios. O gestor regional pode personalizar o branding da plataforma (cores, logo, nome) e gerir conteúdo local (anúncios, jogos nativos, configurações).

A infraestrutura de suporte inclui: tabela `regional_managers` para atribuição de gestores a regiões, tabela `regional_announcements` para comunicados por região, tabela `native_games` para jogos específicos de cada região, e tabela `regional_settings` para configuração de cada região. O componente `RegionalManagerPanel` fornece uma interface com quatro abas: Branding, Configurações, Jogos Nativos e Anúncios. O componente `RegionalCEODashboard` oferece uma visão consolidada para superadmins.

A implementação demonstra maturidade arquitetural com: funções SQL auxiliares para verificar permissões de gestor, políticas RLS que limitam acesso por região, traduções completas nos 6 idiomas para o painel do gestor, e compatibilidade com o sistema de temas dinâmicos. As migrações são idempotentes, permitindo execução segura em múltiplos ambientes. A principal melhoria sugerida é a adição de auditoria de ações do gestor regional, registrando quem alterou o quê e quando.

10. Recomendacoes Prioritarias

10.1 Prioridade Imediata (1-2 semanas)

Implementar uma camada de API server-side para todas as operacoes financeiras e de jogos que envolvem dinheiro real. O Supabase Edge Functions e a opcao mais rapida, pois permite manter a infraestrutura actual e adicionar validacao server-side sem migrar para um backend separado. As Edge Functions devem validar montantes, verificar saldos, aplicar limites de taxa, e registar auditoria antes de qualquer operacao de escrita na base de dados.

Configurar politicas RLS e triggers para proteger as tabelas financeiras (wallet, transactions, bets) contra manipulacao directa. Isto inclui CHECK constraints para impedir saldos negativos, triggers para registar todas as modificacoes na tabela wallet, e funcoes de verificacao de saldo para operacoes de aposta. Enquanto a API server-side nao estiver pronta, as RLS sao a unica defesa contra manipulacao de dados.

10.2 Prioridade Curto Prazo (2-6 semanas)

Converter as importacoes de jogos no App.tsx para importacao dinamica com React.lazy e Suspense. Os 90+ componentes de jogos representam a maior porcao do bundle inicial e nao precisam ser carregados ate que o utilizador aceda a pagina de jogos. Esta alteracao pode reduzir o tamanho do bundle inicial em 50% ou mais, melhorando significativamente o tempo de carregamento.

Expandir a cobertura de traducao para as areas criticas: wallet, pagamentos, e paginas de administracao. A traducao do componente WalletDashboard e dos componentes de deposito/levantamento deve ser prioridade zero, seguida pelas 18 paginas de admin. O sistema de traducao ja esta implementado e funcionando, faltando apenas o trabalho de extrair strings e adicionar chaves de traducao.

10.3 Prioridade Medio Prazo (1-3 meses)

Implementar testes automatizados para os componentes criticos. A plataforma actualmente nao possui ficheiros de teste, o que e um risco significativo para uma aplicacao desta complexidade. Priorizar testes para: AuthContext (fluxos de login, signup, resolucao de roles), ProtectedRoute (verificacao de acesso por role), SafeGameLoader (recuperacao de erros), e componentes de pagamento. Ferramentas recomendadas: Vitest para testes unitarios e Playwright para testes end-to-end.

Refactorar a hierarquia de provedores no App.tsx. A profundidade de sete niveis de providers pode ser reduzida atraves da composicao de contextos ou de um provedor de configuracao unificada. Alem disso, a pagina de erro do AppErrorBoundary mostra texto em portugues endurecido e deve usar o sistema de traducao.

Prioridade	Accao	Impacto	Esforco
Imediata	API server-side (Edge Functions)	Seguranca critica	Alto
Imediata	RLS e triggers financeiros	Seguranca critica	Medio
Curto prazo	Code splitting de jogos	Desempenho +50%	Baixo
Curto prazo	Traducao wallet/pagamentos	Usabilidade	Medio
Medio prazo	Testes automatizados	Confiabilidade	Alto
Medio prazo	Refactor providers	Manutenibilidade	Medio
Medio prazo	Auditoria de gestores	Governanca	Baixo

Tabela 5: Plano de accao prioritario

11. Conclusao

A plataforma Bateu demonstra uma visao de produto ambiciosa e uma capacidade de execucao notavel. Com mais de 200 ficheiros de codigo, 90+ componentes de jogos ao vivo, suporte a 6 idiomas, 5 perfis de utilizador, e operacao multi-regional, a plataforma esta entre as mais completas do mercado de entretenimento digital africano. A qualidade dos jogos, a adaptacao cultural com jogos como Djikota e Ntchuva, e o sistema de gestores regionais sao diferenciais competitivos claros.

No entanto, a plataforma enfrenta desafios significativos que precisam de atencao. A ausencia de uma camada de API server-side e o risco mais critico, especialmente considerando que a plataforma opera com dinheiro real atraves de carteiras digitais e jogos com apostas. O uso de localStorage para dados de registo e referral, embora funcional, nao e ideal para dados sensiveis. A cobertura de traducao de apenas 13% limita o alcance geografico apesar do sistema de i18n estar implementado.

As recomendacoes desta auditoria estao organizadas por prioridade e impacto. A implementacao das accoes de prioridade imediata (API server-side e RLS financeira) deve ser tratada como um requisito de lancamento, nao como uma melhoria futura. As accoes de curto e medio prazo podem ser executadas de forma iterativa, com cada ciclo de sprint abordando uma ou duas recomendacoes. Com a execucao deste plano, a plataforma estara posicionada para escalar de forma segura e sustentavel.